

Focused revision sheet for SQL interview questions and the binary tree spiral level order traversal theory question.

1. Two Students With Closest SAT Scores

Return the two students with the closest score difference. If multiple pairs have the same minimum difference, select the student name combination that is higher in the alphabet.

```
SELECT
    s1.student AS one_student,
    s2.student AS other_student,
    ABS(s1.score - s2.score) AS score_diff
FROM scores s1
JOIN scores s2
ON s1.id < s2.id
ORDER BY
    score_diff ASC,
    one_student DESC,
    other_student DESC
LIMIT 1;
```

Pattern: self join all unique pairs, compute absolute difference, sort by smallest difference.

For the sample, Alice and Scott have scores 2010 and 2100, difference 90.

2. Subscription Date Range Overlap

Return true or false whether each user has a completed subscription date range that overlaps with any other completed subscription.

```
SELECT
    s1.user_id,
    CASE
        WHEN COUNT(s2.user_id) > 0 THEN 1
        ELSE 0
    END AS overlap
FROM subscriptions s1
LEFT JOIN subscriptions s2
ON s1.user_id <> s2.user_id
AND s1.end_date IS NOT NULL
AND s2.end_date IS NOT NULL
AND s1.start_date <= s2.end_date
AND s2.start_date <= s1.end_date
GROUP BY s1.user_id
ORDER BY s1.user_id;
```

Overlap condition:

```
s1.start_date <= s2.end_date
AND s2.start_date <= s1.end_date
```

Pattern: two intervals overlap when each starts before the other ends.

3. Top 3 Highest Employee Salaries By Department

Return full employee name, department name, and salary. If a department has fewer than 3 employees, return all employees from that department.

```
WITH ranked_employees AS (
    SELECT
        CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
        d.name AS department_name,
        e.salary,
        ROW_NUMBER() OVER (
            PARTITION BY d.id
            ORDER BY e.salary DESC
        ) AS rn
    FROM employees e
    JOIN departments d
    ON e.department_id = d.id
)
SELECT
    employee_name,
    department_name,
    salary
FROM ranked_employees
WHERE rn <= 3
ORDER BY department_name ASC, salary DESC;
```

Pattern: use ROW_NUMBER() partitioned by department.

4. Number Of Upsold Customers

Upsold customers are users who bought something on a date after their first purchase date. Purchases on the same day as the first purchase do not count.

```
WITH first_purchase AS (
  SELECT
    user_id,
    MIN(DATE(created_at)) AS first_purchase_date
  FROM transactions
  GROUP BY user_id
)
SELECT
  COUNT(DISTINCT t.user_id) AS num_of_upsold_customers
FROM transactions t
JOIN first_purchase fp
ON t.user_id = fp.user_id
WHERE DATE(t.created_at) > fp.first_purchase_date;
```

Pattern: find each user's first purchase date, then count users with later purchases.

5. Monthly Report For 2020

Show number of users, number of transactions, and total order amount per month in 2020.

```
SELECT
  MONTH(t.created_at) AS month,
  COUNT(DISTINCT t.user_id) AS num_customers,
  COUNT(t.id) AS num_orders,
  SUM(t.quantity * p.price) AS order_amt
FROM transactions t
JOIN products p
ON t.product_id = p.id
WHERE t.created_at >= '2020-01-01'
AND t.created_at < '2021-01-01'
GROUP BY MONTH(t.created_at)
ORDER BY month;
```

Pattern: filter year first, join products for price, group by month.

6. First Touch Attribution For Converted Users

First touch attribution is the channel from the user's earliest session, but only for users who converted at least once.

```
WITH converted_users AS (
  SELECT DISTINCT us.user_id
  FROM user_sessions us
  JOIN attribution a
  ON us.session_id = a.session_id
  WHERE a.conversion = 1
),
ranked_sessions AS (
  SELECT
    us.user_id,
    a.channel,
    ROW_NUMBER() OVER (
      PARTITION BY us.user_id
      ORDER BY us.created_at ASC
    ) AS rn
  FROM user_sessions us
  JOIN attribution a
  ON us.session_id = a.session_id
  JOIN converted_users cu
  ON us.user_id = cu.user_id
)
SELECT
  channel,
  user_id
FROM ranked_sessions
WHERE rn = 1
ORDER BY user_id;
```

Pattern: identify converted users first, then select each converted user's earliest session channel.

7. Sum Salaries Of Employees With No Completed Projects

Find employees who were assigned at least one project and completed none of their projects. A completed project has End_dt IS NOT NULL.

```
WITH slack_employees AS (
  SELECT
    e.id,
```

```

        e.salary
    FROM employees e
    JOIN projects p
    ON e.id = p.employee_id
    GROUP BY e.id, e.salary
    HAVING COUNT(p.project_id) >= 1
    AND SUM(CASE WHEN p.End_dt IS NOT NULL THEN 1 ELSE 0 END) = 0
)
SELECT
    SUM(salary) AS total_slack_salary
FROM slack_employees;

```

Pattern: group by employee, keep employees with zero completed projects, then sum salaries.

8. Spiral Level Order Traversal Of Binary Tree - Theory

Spiral level order traversal is also called zigzag level order traversal.

It visits nodes level by level like BFS, but alternates direction at each level:

- Level 1: left to right
- Level 2: right to left
- Level 3: left to right
- and so on

Interview Explanation

Use a queue for level order traversal. For each level, process all nodes currently in the queue. Store the current level's values in a list. If the direction is left to right, add each value at the end of the list. If the direction is right to left, add each value at the front of the list. After completing one level, toggle the direction.

This gives spiral order because each level is printed in the opposite direction from the previous level.

Java Code

```

import java.util.*;

class Solution {
    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();

        if (root == null) {
            return result;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        boolean leftToRight = true;

        while (!queue.isEmpty()) {
            int size = queue.size();
            LinkedList<Integer> level = new LinkedList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();

                if (leftToRight) {
                    level.addLast(node.val);
                } else {
                    level.addFirst(node.val);
                }

                if (node.left != null) {
                    queue.add(node.left);
                }

                if (node.right != null) {
                    queue.add(node.right);
                }
            }

            result.add(level);
            leftToRight = !leftToRight;
        }

        return result;
    }
}

```

Time: O(n)

Space: $O(n)$

SQL Pattern Summary

- Closest pair: self join + `ABS( )` + `ORDER BY`
- Date overlap: `start1 <= end2 AND start2 <= end1`
- Top N per group: `ROW_NUMBER( ) OVER (PARTITION BY ... ORDER BY ...)`
- Upsold customers: first purchase date + later purchase check
- Monthly report: filter year + group by month
- First touch attribution: converted users + earliest session per user
- Slack salary: group by employee + `HAVING` zero completed projects